

Aula 09 - Busca por ID em lista de objetos

Contexto de negócio

Atendimento ao cliente: localizar conta por identificador sem depender de posicao fisica na lista.

Como encontrar uma conta especifica sem depender da posicao da lista?

Tensão operacional

`get(índice)` e frágil porque a ordem pode mudar.

Retomada necessária: o que é `ArrayList<Conta>`

Na aula 08 usamos uma lista, mas ainda não detalhamos sua anatomia. Antes de buscar uma conta, precisamos entender a estrutura que será percorrida.

O papel do `import`

```
import java.util.ArrayList;
```

`ArrayList` não pertence ao conjunto de classes que o Java disponibiliza automaticamente em todo arquivo. Ele está no pacote `java.util`, um espaço de organização que reúne classes utilitárias, como coleções e ferramentas de entrada.

O `import` permite escrever `ArrayList` no código sem repetir o nome completo:

```
java.util.ArrayList<Conta> contas = new java.util.ArrayList<>();
```

As duas formas representam a mesma classe. O `import` é uma forma de tornar o código mais legível; ele não cria a lista e não executa a busca.

O papel dos sinais `< >`

```
ArrayList<Conta> contas = new ArrayList<>();
```

`ArrayList` é a estrutura da lista. `Conta` entre `< >` é o tipo de elemento que a lista aceita. Essa informação é chamada de *generics*.

Leia a declaração assim:

`contas` é uma lista dinâmica que só deve guardar objetos `Conta`.

O `<>` não é comparação matemática e não é um operador de tamanho. Ele cria um contrato de tipo para a coleção. Por isso este código é aceito:

```
contas.add(new Conta(4, "Diego", 900.00));
```

E este código deve ser rejeitado pelo compilador:

```
contas.add("nao e uma conta");
```

Na criação, `new ArrayList<>()` usa o chamado operador diamante: o compilador deduz `Conta` a partir do lado esquerdo. Para enxergar a mesma ideia sem essa abreviação, podemos escrever `new ArrayList<Conta>()`.

Array comum e `ArrayList`

As duas estruturas guardam vários valores, mas têm contratos diferentes:

```
Conta[] contasFixas = new Conta[2];
contasFixas[0] = new Conta(1, "Ana", 1000.00);
contasFixas[1] = new Conta(2, "Bruno", 700.00);
```

O array nasce com tamanho definido. Para guardar uma terceira conta, é preciso criar outro array e copiar os elementos, ou prever capacidade sobrando.

```
ArrayList<Conta> contasDinamicas = new ArrayList<>();
contasDinamicas.add(new Conta(1, "Ana", 1000.00));
contasDinamicas.add(new Conta(2, "Bruno", 700.00));
contasDinamicas.add(new Conta(3, "Carla", 500.00));
```

Na lista, `add` aumenta a coleção conforme o cadastro cresce. O tamanho pode ser consultado com `size()`, e a lista pode ser testada com `isEmpty()`.

Necessidade	Array	ArrayList
Criar estrutura	<code>new Conta[2]</code>	<code>new ArrayList<Conta>()</code>
Adicionar	atribuir por índice	<code>add(conta)</code>
Acessar posição	<code>contas[0]</code>	<code>contas.get(0)</code>
Remover	exige reorganização manual	<code>remove(indice)</code> ou <code>remove(objeto)</code>
Tamanho	<code>length</code> fixo	<code>size()</code> atualizável
Verificar vazio	comparar tamanho	<code>isEmpty()</code>

O `ArrayList` não elimina o índice: `get(0)` ainda significa “primeiro item”. Ele resolve o crescimento e as operações da coleção, mas não transforma a posição em identidade de negócio. É exatamente por isso que esta aula ainda precisa resolver a busca por `id`.

Percorrer sem confundir posição com identidade

```
for (Conta conta : contas) {  
    System.out.println(conta);  
}
```

O `for-each` entrega cada objeto da coleção, um por vez. Ele é adequado quando queremos examinar todas as contas e não precisamos controlar o índice. A busca por ID desta aula usará essa forma de percurso e comparará um dado da conta, não a posição em que ela apareceu.

Vocabulário mínimo da coleção

- `add(conta)`: acrescenta um objeto ao final da lista.
- `get(indice)`: devolve o objeto daquela posição; índice começa em zero.
- `size()`: informa quantos objetos existem neste momento.
- `isEmpty()`: informa se não há objetos.
- `remove(...)`: retira um elemento, assunto que será usado no fluxo posterior.

Teste rápido antes da busca:

```
System.out.println(contas.size());  
System.out.println(contas.isEmpty());  
System.out.println(contas.get(0).titular);
```

Com três contas, a saída é `3`, depois `false`, depois `Ana`. O aluno deve conseguir prever esses valores antes de executar.

Tentativa que ainda não resolve

O código da aula 08 consegue operar Bruno com `contas.get(1)`. Isso parece simples, mas mistura duas ideias diferentes: a posição usada pela lista e a identidade da conta.

Se Ana for removida ou se a ordem mudar, o mesmo `get(1)` pode operar outra conta. A operação continua compilando, mas o resultado de negócio pode estar errado.

Pergunta central:

- como localizar a conta 2 mesmo quando a posição dela mudar?

Objetivo técnico da entrega

- Busca sequencial por ID.
- Retorno de objeto.

- Retorno `null` quando não encontra.
- Explicar por que buscar por posição deixa de ser sustentável.
- Demonstrar impacto prático de buscar por identificador estável.

Recurso técnico desta aula

Método central desta etapa:

```
static Conta buscarPorId(ArrayList<Conta> p_contas, int p_id)
```

Fundamento novo: identificador estável vs posição da lista

Esta aula introduz uma mudança de pensamento operacional:

- posição da lista é detalhe estrutural;
- ID da conta é identidade de negócio.

Quando a ordem da lista muda, o índice muda. Quando a identidade da conta não muda, o ID permanece.

Pergunta de condução:

- em atendimento real, a pessoa diz "sou o índice 1" ou informa um identificador da conta?

Problema que justifica o novo artefato

Operar por posição em lista é frágil: a ordem muda e a conta-alvo pode ser outra. Atendimento precisa de critério estável de localização.

Antes e depois da mudança

Antes (dependência de posição):

```
Conta conta = contas.get(1);
```

Problema:

- se a lista mudar de ordem, `get(1)` pode apontar para outra conta.

Depois (dependência de identidade):

```
Conta conta = buscarPorId(contas, 2);
```

Ganho real:

- o fluxo passa a buscar por regra de negócio (ID), não por detalhe estrutural (posição).

Artefato explicado: busca por ID com retorno de objeto

Artefato desta aula: `buscarPorId(...)`.

- percorre a lista sequencialmente;
- retorna `Conta` quando encontra;
- retorna `null` quando não encontra.

Esse contrato obriga o chamador a tratar ausência de resultado antes de operar.

Leitura linha a linha da ideia do algoritmo:

1. Percorrer toda a coleção (`for-each`).
2. Comparar `conta.id` com o `p_id` solicitado.
3. Retornar imediatamente quando encontra.
4. Retornar `null` quando termina sem encontrar.

Esse desenho é uma busca sequencial simples, previsível e legível para a etapa da trilha.

Demonstração prática do impacto

Cenário A (posição):

- lista original: `[Ana(id 1), Bruno(id 2), Carla(id 3)]`;
- `get(1)` retorna Bruno.

Cenário B (reordenação):

- lista reordenada: `[Carla(id 3), Ana(id 1), Bruno(id 2)]`;
- `get(1)` agora retorna Ana.

Cenário C (busca por ID):

- `buscarPorId(contas, 2)` retorna Bruno em ambos os cenários.

Conclusão:

- índice é frágil para operação de negócio;
- ID é estável para operação de negócio.

Limites da solução atual

Importante não romantizar a solução:

- busca sequencial tem custo linear (varre lista até encontrar);
- funciona bem no contexto didático e em volumes pequenos/médios;
- para volumes muito grandes, surgem outras estratégias de acesso.

Pergunta de engenharia para maturidade:

- quando simplicidade e legibilidade valem mais que otimização estrutural?

Transição didática para a próxima aula

Com busca estável, o próximo passo é organizar as operações em camada de serviço para reduzir dispersão no main.

Valor prático entregue

- Reduz erro de atendimento por busca incorreta.
- A base fica pronta para evolução controlada da próxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.
- Executar a regra principal e observar o impacto no estado.
- Operar a coleção de contas com validação de alvo e consistência de dados.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

Código em foco

```
import java.util.ArrayList;

class Conta {
    int id;
    String titular;
    double saldo;

    Conta(int p_id, String p_titular, double p_saldoInicial) {
        id = p_id;
        titular = p_titular;
        saldo = p_saldoInicial;
    }

    void depositar(double p_valor) {
        if (p_valor > 0) {
            saldo = saldo + p_valor;
        }
    }

    boolean sacar(double p_valor) {
        if (p_valor > 0 && p_valor <= saldo) {
            saldo = saldo - p_valor;
            return true;
        }
        return false;
    }

    @Override
    public String toString() {
        return "ID: " + id
            + " | Titular: " + titular
            + " | Saldo: " + String.format("%.2f", saldo);
    }
}
```

```
// Classe temporaria de fluxo do programa.
// Neste curso, ela existe para iniciar a execucao em Java.
public class AppGestaoContas {
    public static void main(String[] args) {
        // Aula 09: busca por ID em uma colecao de objetos.
        ArrayList<Conta> contas = new ArrayList<>();
        contas.add(new Conta(1, "Ana", 1000.00));
        contas.add(new Conta(2, "Bruno", 700.00));
        contas.add(new Conta(3, "Carla", 500.00));

        System.out.println("=== LISTA INICIAL ===");
        listarContas(contas);

        Conta contaEncontrada = buscarPorId(contas, 2);
        if (contaEncontrada != null) {
            System.out.println("\nConta encontrada: " + contaEncontrada);
            contaEncontrada.depositar(100.00);
        }

        Conta contaInexistente = buscarPorId(contas, 99);
        if (contaInexistente == null) {
            System.out.println("\nConta 99 nao encontrada.");
        }

        System.out.println("\n=== LISTA APOS DEPOSITO ===");
        listarContas(contas);
    }

    static Conta buscarPorId(ArrayList<Conta> p_contas, int p_id) {
        for (Conta conta : p_contas) {
            if (conta.id == p_id) {
                return conta;
            }
        }
        return null;
    }

    static void listarContas(ArrayList<Conta> p_contas) {
        for (Conta conta : p_contas) {
            System.out.println(conta);
        }
    }
}
```

Leitura técnica orientada

Percorra o código com estes checkpoints de revisão:

- Estrutura de dominio: identificar onde a entidade de negócio concentra dados.
- Regra de decisão: mapear condições que aprovam ou negam operações.
- Estrutura de dados: justificar uso de lista dinamica no contexto da aula.

- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.
- Identidade de negócio: diferenciar claramente índice de lista e identificador da conta.

Discussão de contrato de retorno

Quando o método retorna `boolean`, o contrato precisa ser tratado como regra de negócio:

- `true` significa operação aprovada e estado atualizado;
- `false` significa operação negada e estado preservado.

Risco de lógica silenciosa:

- ignorar o `false` no chamador cria fluxo aparentemente correto, mas operacionalmente incompleto.

Responsabilidade de quem chama:

- sempre tratar os dois ramos (`true` e `false`) com mensagens e ações coerentes;
- registrar motivo de negativa quando possível (conta inativa, valor inválido, limite excedido);
- evitar seguir o fluxo como se a operação tivesse ocorrido.

Variações para debate técnico:

1. Em quais cenários `boolean` basta?
2. Quando o time precisa de retorno com motivo da falha?
3. Qual custo de não tratar o `false` em produção?

Contrato adicional: retorno `null`

Quando a busca retorna `null`, o contrato também exige disciplina:

- `null` significa "não encontrado", não "erro de execução";
- o chamador deve interromper a operação dependente e comunicar o motivo.

Erro comum a prevenir:

- seguir com depósito/saque/remoção sem validar `null`, gerando falhas de lógica ou exceções.

Perguntas de decisão

1. Por que `get(1)` é frágil?
2. O que significa retorno `null`?
3. Por que o método retorna `Conta`?
4. O que muda no fluxo quando buscamos por ID e não por posição?
5. Qual risco de seguir operação sem validar `null`?
6. Em que cenário esta busca sequencial deixaria de ser suficiente?

Variações de cenário

1. Buscar ID inexistente.
2. Trocar ordem das contas na lista.
3. Buscar o mesmo ID antes e depois de reordenar a lista e comparar resultado.

Exercícios progressivos

1. Criar método `existeConta` retornando boolean.
2. Criar método `depositarEmConta` usando `buscarPorId`.
3. Criar `sacarDeContaPorId` com tratamento explícito para `null`.
4. Registrar em comentário: "qual parte do fluxo protege contra conta não encontrada?".

Desafio de equipe

Mostrar no terminal cada conta analisada durante a busca.

Critérios de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.
- Código permanece legível e coerente com o nível da aula.

Checklist de progressividade técnica

- Pré-requisito confirmado: leitura e execução do código-base da aula anterior.
- Novidade técnica identificada: explicar em uma frase o recurso novo desta aula.
- Complexidade controlada: apontar qual decisão de projeto evita acoplamento desnecessário.
- Evidência prática: executar ao menos um caso de sucesso e um caso de erro no Tryit.

Fechamento executivo

Agora conseguimos localizar conta por identificacao. Na próxima, vamos organizar operações sem depender de menu.

Revisão rápida (5 minutos)

1. O que mudou ao sair de `get(indice)` para `buscarPorId`?
2. Qual erro de negócio essa mudança evita?
3. Qual é o contrato exato do retorno `null`?
4. Evolução: qual pequena extensão agregaria mais valor sem aumentar acoplamento?

Mini-missao de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

Execução no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler
2. Colar todo o conteúdo de `AppGestaoContas.java` no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercícios.
4. Manter uma aba separada para cada exercício e comparar resultados.

Referências W3Schools da aula

- https://www.w3schools.com/java/java_loops.asp

- https://www.w3schools.com/java/java_methods.asp
- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>